

ThoughtLens Build Tracking Document

This document breaks the project down from 0% to 100% into concrete, trackable phases so the product is built as a real, working system instead of a half-finished prototype. The plan is grounded in Anthropic's March 2025 circuit tracing research, which uses sparse interpretable features and attribution graphs to trace how a language model reaches an output.[web:21] Their companion case-study paper on Claude 3.5 Haiku shows that mechanisms like multi-hop reasoning, rhyme planning, hallucination, multilingual abstraction, and refusal can be studied at the circuit level rather than through surface behavior alone.[web:4][web:21]

The package also has to sit clearly apart from existing interpretability libraries such as TransformerLens and Anthropic-aligned circuit tracing tooling, both of which already occupy nearby space in the ecosystem.[web:22][web:30] That is why the build plan below treats **platform-agnostic support**, **real probes**, and a **serious UI layer** as first-class deliverables rather than extras.[web:21][web:30]

Success definition

A 100% complete ThoughtLens release means four things are true at once: users can install the package from PyPI, connect it to multiple model backends, generate interpretable traces and reports that are actually useful, and trust the outputs because the system has tests, benchmarks, and explicit limitations documented.[web:17][web:21] It also means the project avoids confusion with existing packages and names on PyPI, where naming collisions are a known practical problem.[web:17][web:20]

Progress model

Range	Meaning
0–10%	Project exists only as validated scope, research grounding, and repo skeleton.
10–25%	Core architecture runs locally with mocks and deterministic tests.
25–45%	Real providers and base tracing pipeline work end-to-end.
45–65%	White-box interpretability and black-box probing are both functional.
65–80%	UI/reporting layer becomes genuinely useful for researchers.
80–90%	Probe suite is complete, benchmarked, and validated.
90–100%	Packaging, docs, CI, release engineering, and usability polish are complete.

Phase 0 — Foundation and control plane (0% → 10%)

This phase is complete only when the project is tightly scoped and every major design choice is frozen in writing before feature work begins. Anthropic's work is specifically about *mechanistic* interpretability through sparse feature dictionaries and attribution graphs, so this phase must lock the project to that target and explicitly reject shallow substitutes like raw attention maps marketed as "thinking."^[web:21]^[web:4]

Deliverables

- Final project name selected and reserved for package/repo/domain direction.
- Repository created with branch strategy, issue templates, labels, and milestone structure.
- Product definition frozen: provider matrix, feature list, success metrics, and non-goals.
- Research source map created from the March 2025 attribution graph paper and biology paper.^[web:4]^[web:21]
- Security baseline created for PyPI publishing because PyPI package confusion and supply-chain issues are a real risk area.^[web:23]^[web:29]

Tasks

1. Choose and lock the package name.
2. Create GitHub repo, default branch protections, CODEOWNERS, issue templates, PR template.
3. Create milestone board with phases matching this document.
4. Write README stub, LICENSE, CONTRIBUTING, SECURITY, CODE_OF_CONDUCT.
5. Freeze architecture decisions in ADRs (Architecture Decision Records).
6. Define performance budget, API budget, and minimum supported Python version.

Exit criteria

- Repo is public or ready to go public.
- Name is final.
- Every subsequent phase has tickets.
- No code beyond scaffolding is written before this is done.

Tracking checklist

- ☐ Name locked.
- ☐ Repo created.
- ☐ Milestones created.
- ☐ Security policy added.
- ☐ ADR-001 through ADR-005 written.

Phase 1 — Package skeleton and local developer workflow (10% → 20%)

This phase turns the idea into a runnable Python project with repeatable local setup. The goal is not features yet; it is to ensure every future feature lands into a stable package structure with linting, formatting, typing, and tests already enforced.[web:17]

Deliverables

- Installable package skeleton with `pyproject.toml`.
- Working editable install: `pip install -e .`
- Lint, format, type-check, and test commands all wired.
- CI workflow running on pushes and pull requests.
- Mock provider and deterministic test fixture available.

Tasks

1. Create package tree:

- `thoughtlens/`
- `providers/`
- `features/`
- `circuits/`
- `probes/`
- `visualization/`
- `utils/`
- `tests/`
- `examples/`

2. Add tooling:

- Ruff
- Black
- MyPy or Pyright
- Pytest
- Coverage
- Pre-commit hooks

3. Add GitHub Actions:

- lint
- typecheck
- tests on Python 3.10/3.11/3.12

4. Build `MockProvider` returning deterministic activations and logits.

5. Add golden-file testing strategy for graphs and reports.

Exit criteria

- `make lint`, `make test`, and `make typecheck` all pass locally and in CI.
- Package installs cleanly.
- Mock-based smoke test traces a prompt without any external API key.

Tracking checklist

- ☐ `pyproject.toml` complete.
- ☐ Editable install works.
- ☐ Pre-commit enabled.
- ☐ CI green.
- ☐ Mock provider test green.

Phase 2 — Provider layer done for real (20% → 35%)

This phase delivers the real model connectivity layer. Anthropic's papers are based on deep internal access, but a usable product must support both **white-box** models with real activations and **black-box** API models where only outputs and top-token behavior are observable.^[web:4]
^[web:21]

Deliverables

- `BaseProvider` contract finalized.
- OpenAI provider functional in black-box mode.
- Anthropic provider functional in black-box mode.
- HuggingFace provider functional with hidden states and attentions.
- Ollama provider functional for local models.
- Unified `ProviderOutput` returned across all providers.

Tasks

1. Finalize normalized schema:

- `prompt`
- `tokens`
- `token_ids`
- `activations`
- `attentions`
- `logits`
- `top_tokens`

- meta

2. Implement OpenAI adapter using token/logprob APIs where available.
3. Implement Anthropic adapter using messages API and black-box-compatible outputs.
4. Implement HuggingFace adapter using transformers with `output_hidden_states=True` and `output_attentions=True`.
5. Implement Ollama adapter using local REST interface.
6. Add integration tests behind env vars.
7. Add caching/retry behavior and clear error surfaces.

Exit criteria

- A single tracing call can run against all four providers using the same high-level API.
- HuggingFace provider returns real arrays for hidden states and attentions.
- Closed-model providers degrade gracefully without pretending they have internal access.

Tracking checklist

- [] OpenAI provider live-tested.
- [] Anthropic provider live-tested.
- [] HuggingFace provider live-tested.
- [] Ollama provider live-tested.
- [] Unified schema stable.

Phase 3 — Minimal end-to-end tracing pipeline (35% → 45%)

This is the first time the product becomes visibly real. The target is a prompt going in and a trace artifact coming out: tokens, top features, an attribution graph, and a basic report. It does not need to be perfect yet, but it must be real and end-to-end.[web:21]

Deliverables

- `Scope.trace()` works end-to-end.
- `TraceResult` object is stable.
- Graph object can serialize to JSON.
- HTML report v0 works.
- CLI trace command works.

Tasks

1. Build `Scope` entry point.
2. Build `TraceResult` data object.
3. Build initial `FeatureExtractor`:
 - white-box path: simple PCA/logit-lens baseline
 - black-box path: token masking importance baseline
4. Build initial `CircuitTracer`:
 - input nodes
 - feature nodes
 - output node
 - weighted edges
5. Build graph prune and top-path logic.
6. Build HTML report v0 with:
 - output token
 - top alternatives
 - top features
 - graph embed
7. Build CLI:
 - `thoughtlens trace --provider ...`

Exit criteria

- User can run one command and get a working report.
- Output is not fake: every number in the report comes from the actual trace pipeline.

Tracking checklist

- [] `Scope.trace()` stable.
- [] Graph serializes.
- [] CLI works.
- [] HTML report opens standalone.
- [] End-to-end smoke demo recorded.

Phase 4 — Black-box interpretability engine that is honest and useful (45% → 55%)

Closed models cannot expose real internal activations through public APIs, so this phase builds the best honest approximation rather than faking internal visibility. That aligns with the user's requirement to show what the model is doing **without pretending** the system sees hidden states it does not actually have.[web:4][web:21]

Deliverables

- Token masking importance engine.
- Pairwise token interaction scoring for multi-hop prompts.
- Budget-aware API execution planner.
- Confidence and uncertainty annotations.
- Clear black-box labeling in UI/report so users never mistake approximation for full mechanistic access.

Tasks

1. Build baseline token ablation loop.
2. Add pairwise interaction mode for selected prompts.
3. Add caching layer for repeat perturbations.
4. Add API cost estimator.
5. Add uncertainty score for each causal claim.
6. Add UI/report language:
 - "Observed via perturbation"
 - "Inferred interaction"
 - "Not directly observed internally"

Exit criteria

- API-backed reports are useful, reproducible, and explicit about limits.
- The tool never labels black-box token perturbation as direct neuron-level observation.

Tracking checklist

- [] Token ablation live-tested.
- [] Pairwise interaction works.
- [] Cache works.
- [] Budget guard works.
- [] Report language clearly marks inference vs observation.

Phase 5 — White-box mechanistic core with SAE training (55% → 70%)

This is the heart of the real research-grade product. Anthropic's methods paper is built around interpretable sparse features that replace polysemantic MLP outputs with feature dictionaries, enabling attribution graphs over concepts rather than raw neurons.[web:21] ThoughtLens becomes truly differentiated only when this phase is working end-to-end.[web:22][web:30]

Deliverables

- Activation cache pipeline.
- TopK Sparse Autoencoder implementation.
- SAE training loop that converges on real model activations.
- Feature extraction from SAE codes.
- Feature labeling pipeline.
- Supernode clustering.

Tasks

1. Build activation collection pipeline for HuggingFace models.
2. Write efficient storage format (npz, HDF5, or memmap).
3. Implement TopK-SAE.
4. Add metrics:
 - reconstruction error
 - average L0 sparsity
 - dead feature fraction
 - feature density histogram
5. Add feature extraction from sparse codes.
6. Build auto-labeling system from top activating contexts.
7. Build human override for labels.
8. Build supernode clustering with cosine similarity + label affinity.

Exit criteria

- SAE trains on a real model layer and yields usable features.
- Extracted features are visible in traces and reports.
- At least one case study (for example Dallas→Texas→Austin) is shown with interpretable feature nodes rather than generic PCA directions.[web:21]

Tracking checklist

- [] Activation cache complete.
- [] SAE trains.
- [] Metrics logged.
- [] Feature labels generated.
- [] Supernodes generated.
- [] White-box trace demo works.

Phase 6 — Attribution graph quality and path analysis (70% → 78%)

This phase upgrades the graph from "it renders" to "it explains." Anthropic's graph descriptions are compelling because they isolate causal structure instead of dumping every possible connection.[web:21]

Deliverables

- Strong graph pruning logic.
- Path ranking and explanation summaries.
- Suppressor/inhibitor edge support.
- Error residual node support.
- Graph comparison/diff support.

Tasks

1. Improve edge scoring beyond naive similarity.
2. Add indirect edge support.
3. Add suppressor edges and coloring.
4. Add error residual node showing unexplained computation mass.
5. Add top-k path summarizer.
6. Add graph diff between baseline and intervention runs.
7. Add export to JSON and CSV.

Exit criteria

- Top paths are interpretable and compact.
- Graphs do not devolve into spaghetti for ordinary prompts.
- Interventions visibly alter graph structure.

Tracking checklist

- [] Path ranking stable.
- [] Suppressor edges visible.
- [] Error node works.
- [] Graph diff works.
- [] Export formats complete.

Phase 7 — Deep UI layer that actually shows model reasoning (78% → 88%)

This phase is where the product becomes something people will actually use repeatedly. The UI must present the exact distinction the papers emphasize: not generic token saliency, but a layered account of how concepts form and route toward an output token.[web:4][web:21]

Deliverables

- Token heatmap view.
- Attribution graph explorer.
- Residual stream trajectory view.
- Feature browser.
- Probe dashboard.
- Standalone HTML report with tabs.

Tasks

1. Build token heatmap with hover feature list.
2. Build layered Plotly attribution graph.
3. Build residual stream PCA/trajectory animation for white-box runs.
4. Build searchable feature browser table.
5. Build probe scorecard dashboard.
6. Build tabbed self-contained HTML report.
7. Add dark mode and export to PNG/HTML.
8. Add clear UI text distinguishing:
 - direct observation
 - inferred causal effect
 - approximate black-box result

Exit criteria

- A researcher can open one report and understand the model's path to output without reading raw JSON.
- The UI is useful for both black-box and white-box modes, while making their limits obvious.

Tracking checklist

- [] Heatmap done.
- [] Graph explorer done.
- [] Residual stream view done.

- [] Feature browser done.
- [] Probe dashboard done.
- [] HTML report polished.

Phase 8 — Full probe suite mapped to the paper (88% → 94%)

The March 2025 Anthropic case-study paper is not just background reading; it is the blueprint for the benchmark suite.^[web:4] This phase operationalizes those findings into reusable probes.

Deliverables

- Multi-hop probe.
- Capitals probe.
- Rhyme planning probe.
- Persona consistency probe.
- Multilingual abstraction probe.
- Hallucination probe.
- CoT faithfulness probe.
- Refusal/jailbreak anatomy probe.
- Suppressor probe.
- Motivated reasoning probe.

Tasks

1. Implement each probe as a standalone class.
2. Define prompt sets and expected evidence schema.
3. Add pass/fail plus graded score.
4. Add per-probe explanation text.
5. Add benchmark runner and JSON output.
6. Validate probes across at least:
 - one OpenAI model
 - one Anthropic model
 - one HuggingFace model
 - one Ollama model

Exit criteria

- Every probe runs from code and from CLI.
- Probe outputs are comparable across models.
- Results are stable enough to use as a benchmark dashboard.

Tracking checklist

- [] 10/10 probes implemented.
- [] Probe runner stable.
- [] CLI benchmark command added.
- [] Cross-provider validation complete.

Phase 9 — Interventions, comparisons, and truth-testing (94% → 97%)

This phase tests whether the interpretability claims are causal instead of merely descriptive. Anthropic's work becomes convincing because identified features can be manipulated and the downstream computation changes in predictable ways.[web:21]

Deliverables

- Feature inhibit/amplify/clamp support.
- Before/after trace comparison.
- Intervention report mode.
- Side-by-side graph diff UI.

Tasks

1. Add intervention hooks for white-box HuggingFace models.
2. Implement feature clamp in forward pass.
3. Compare baseline vs intervention outputs.
4. Compare baseline vs intervention graphs.
5. Add intervention notebook examples.
6. Add validation suite for known causal cases.

Exit criteria

- At least three known hypotheses can be validated causally.
- Example: suppressing a geographic feature reduces correct capital retrieval in a predictable way.[web:21]

Tracking checklist

- ☐ Feature inhibition works.
- ☐ Feature amplification works.
- ☐ Before/after graph diff works.
- ☐ Causal examples documented.

Phase 10 — Release engineering, docs, and production polish (97% → 100%)

This phase makes the project publishable and maintainable. A real product ends here, not at the first demo.

Deliverables

- Clean PyPI release.
- Complete docs site.
- Examples and tutorials.
- Reproducible benchmark outputs.
- Versioned changelog.
- Release checklist.

Tasks

1. Finalize package metadata and classifiers.
2. Build and publish to TestPyPI, then PyPI.
3. Create docs site with quickstart, concepts, probe reference, SAE guide, API docs.
4. Add screenshots and generated demo reports.
5. Add benchmark examples and expected outputs.
6. Add semantic versioning and release notes process.
7. Add telemetry policy statement (preferably none by default).
8. Run usability pass with 3–5 external testers.

Exit criteria

- `pip install thoughtlens` works from PyPI.
- New user can follow quickstart and generate a report in under 15 minutes.
- Docs, tests, and release tags match the shipped code.

Tracking checklist

- ☐ TestPyPI release.
- ☐ PyPI release.
- ☐ Docs site live.
- ☐ Tutorials complete.
- ☐ Changelog and release checklist complete.

Cross-phase governance

Some work must continue across all phases because it directly affects whether the system is trustworthy and usable.

A. Truthfulness guardrails

The interface must never imply direct internal observation for models where only black-box perturbation is available. Anthropic's own work distinguishes between actual circuit tracing with internal access and broader black-box investigation, and the product must preserve that distinction in both UI and docs.[web:4][web:21]

- ☐ Every result is tagged as observed, inferred, or approximated.
- ☐ Reports explain the exact evidence source.
- ☐ Closed-model runs never show fake hidden-state visualizations.

B. Quality gates

- ☐ Coverage target $\geq 80\%$ for core packages.
- ☐ Integration tests for every provider.
- ☐ Report generation tested in CI with mock provider.
- ☐ Performance budget documented.
- ☐ API budget documented.

C. Security gates

Recent PyPI incidents involving AI-adjacent packages show that supply-chain hygiene is not optional for this class of tooling.[web:23][web:29]

- ☐ Trusted Publishing configured for PyPI.
- ☐ Dependency pinning/constraints file maintained.
- ☐ `pip-audit` or equivalent in CI.
- ☐ Signed tags/releases.

Suggested repo milestone mapping

Milestone	Percent target	Scope
M1	20%	Repo, package, tooling, CI, mocks
M2	35%	Providers complete
M3	45%	End-to-end trace and report
M4	55%	Honest black-box engine
M5	70%	SAE and white-box feature extraction
M6	78%	Graph quality and path analysis
M7	88%	Full deep UI/report layer
M8	94%	Full probe suite
M9	97%	Interventions and diffing
M10	100%	Release, docs, PyPI

First execution order

This is the exact order to start building without drifting:

1. Finish **Phase 0** completely.
2. Build **Phase 1** and refuse to add features until lint/test/typecheck are clean.
3. Build **Phase 2** providers and validate real connectivity.
4. Build **Phase 3** minimal end-to-end trace so there is one working spine.
5. Split into two streams:
 - **Stream A:** Phase 4 black-box engine.
 - **Stream B:** Phase 5 white-box SAE engine.
6. Merge them in **Phase 6** and **Phase 7** through the shared graph/UI layer.
7. Add **Phase 8** probes only after the trace layer is trustworthy.
8. Add **Phase 9** interventions only after the SAE path is stable.
9. Finish **Phase 10** only when all earlier milestone demos are already working.

What to do right now

The next actual build step is not more design; it is to start at **Phase 0** and create the repo control plane and execution board. That is the right first move because the package name, repo structure, milestone map, and security/release posture need to be locked before any more implementation spreads across files and branches.[web:17][web:23]